Beyond Binary: Unified Detection and Attribution of AI-Generated Code via Hierarchical Family-Aware Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large language models now generate code that is virtually indistinguishable from human-written code—binary detectors exceed 99% accuracy, yet the real-world threat has moved far beyond the binary question. Practitioners need to know *which model* produced a snippet, whether a detector can generalize to *unseen languages and domains*, and whether it can withstand *adversarial evasion* by models fine-tuned to mimic human style. We observe that a fundamental obstacle to progress on these harder tasks is the *genealogical confusion* among LLMs: fine-tuned variants (e.g., Nxcode derived from Qwen1.5) inherit stylometric signatures from their parents, causing >40% misclassification.

We propose **HierTreeCode**, a framework that treats LLM genealogy as structured prior knowledge. A *Hierarchical Affinity Loss* encodes parent–child relationships as embedding-space distance constraints, while a multi-stream architecture fuses token semantics, abstract syntax structure, and spectral frequency patterns. We evaluate on three benchmarks spanning 3.6M code samples, 77 models, 9 languages, and four detection scenarios: binary, authorship attribution, out-of-distribution generalization, and adversarial robustness. On the CoDET-M4 benchmark, HierTreeCode achieves **70.55%** macro-F1 on 6-class attribution—**+4.22 points** over UniXcoder and **+5.75 points** over CodeBERT—while closing the gap on the hardest confused classes.

1 Introduction

Code generation by large language models has reached a tipping point. GPT-4o, CodeLlama, DeepSeek-Coder, and their dozens of variants now produce code that passes the same tests as human-written solutions [Orel et al., 2025]. The resulting *binary detection* problem—"is this code human or machine?"—has been effectively solved: the best detectors achieve >98% F1 [Orel et al., 2025, AICD, 2025]. Yet the world has moved on.

The threats that matter are no longer binary. Consider the following scenarios that current detectors cannot address:

- *Supply-chain attribution.* A vulnerability is discovered in open-source code. Was it generated by GPT-4o, CodeLlama, or a fine-tuned variant? Attribution to the specific model family enables targeted auditing and liability analysis.
- *Cross-language deployment.* A detector trained on Python/Java/C++ is deployed to scan Go and Rust repositories. Current methods collapse from 99% to under 30% F1 in this out-of-distribution setting [AICD, 2025].

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Binary detection (human vs. machine) on CoDET-M4. The task is effectively solved—all transformer methods exceed 98% F1. We report macro-averaged P/R/F1/Acc (%).

Method	Prec.	Recall	F1	Acc.
Fast-DetectGPT (zero-shot)	71.09	65.14	62.03	65.17
SVM	72.41	72.35	72.19	72.19
CatBoost	88.71	88.81	88.78	88.79
CodeBERT	95.70	95.72	95.70	95.71
CodeT5	98.36	98.35	98.35	98.35
UniXcoder	98.65	98.66	98.65	98.65
HierTreeCode (ours)	**99.06**	**99.06**	**99.06**	**99.06**

- *Adversarial evasion.* Models fine-tuned with DPO to mimic human coding style evade binary detectors. A new generation of adversarial code generators demands robustness beyond in-distribution evaluation [DROID, 2025].

Why is attribution so hard? We identify a fundamental structural obstacle: *genealogical confusion*. Modern LLMs do not emerge independently—they form family trees. Nxcode is fine-tuned from Qwen1.5; CodeLlama descends from Llama. These parent–child pairs share overlapping stylometric fingerprints, making them nearly indistinguishable to classifiers that treat all labels as equidistant. On CoDET-M4, Qwen1.5 achieves only 0.44 F1—barely above random chance for a 6-class task—because over 40% of its samples are misclassified as Nxcode.

Our insight. The family tree of LLMs is not a nuisance—it is *structured prior knowledge* that should guide learning. Models within the same family should be closer in embedding space than models from different families, but still separable. We encode this constraint as a *Hierarchical Affinity Loss* that operates on cosine distances with batch-hard triplet mining.

HierTreeCode combines this loss with a multi-stream architecture that captures three complementary views of code authorship: (i) token-level semantics via a pretrained transformer, (ii) syntactic structure via AST sequence encoding, and (iii) frequency-domain patterns via multi-scale spectral analysis of token sequences. We evaluate across three benchmarks totaling 3.6M samples:

- **CoDET-M4** [Orel et al., 2025]: 500K samples, 5 generators, binary + 6-class attribution, per-language/source/generator breakdown.
- **AICDBench** [AICD, 2025]: 2.05M samples, 77 models, 9 languages, progressive OOD evaluation across unseen languages and domains.
- **DroidCollection** [DROID, 2025]: 1.06M samples, machine-refined code and DPO-tuned adversarial examples.

On CoDET-M4, HierTreeCode achieves **70.55%** macro-F1 on author attribution, improving over UniXcoder by +4.22 points and over CodeBERT by +5.75 points. The largest gains come on the hardest classes: Qwen1.5 F1 improves by +3.02 points over the ablation baseline. We release code and all experiment configurations.

2 The Case for Going Beyond Binary

To motivate our approach, we first characterize the landscape of AI code detection across three axes of difficulty, using established benchmarks as evidence.

Binary detection is ceiling-bound. Table 1 shows that even a gradient booster (CatBoost) reaches 88.78% F1 on CoDET-M4 binary detection, and fine-tuned transformers exceed 98%. With margins of improvement under 1%, the binary task no longer discriminates between methods. The research frontier must shift.

Attribution reveals the real challenge. When the task shifts from "is this AI?" to "which AI?", performance drops dramatically. Table 2 shows that UniXcoder—the strongest paper baseline—achieves only 66.33% F1 on 6-class attribution, a 32-point drop from binary. This is where methods must innovate.

OOD generalization is catastrophic. AICDBench reveals that models achieving 99.5% validation F1 on binary detection collapse to <30% test F1 when evaluated on unseen languages and domains [AICD, 2025]. CoDET-M4's unseen-language evaluation (Table 5) shows the same pattern: even UniXcoder drops from 98.65% to 88.96% in the best case.

Adversarial code is the next frontier. DroidCollection introduces DPO-tuned models that generate code specifically designed to mimic human style. Standard detectors show reduced performance on these adversarial samples, demanding methods that capture deeper stylometric invariants [DROID, 2025].

3 HierTreeCode

Our method addresses the three challenges above through two key design decisions: (1) a multi-stream architecture that captures complementary authorship signals, and (2) a hierarchical loss that encodes LLM genealogy as structured supervision.

3.1 Multi-Stream Code Representation

Token stream. We encode source code with ModernBERT-base [ModernBERT, 2024] using SDPA attention, extracting the `[CLS]` representation $h_{\text{tok}} \in \mathbb{R}^{768}$. This captures semantic and lexical patterns.

Structure stream. We parse code into abstract syntax trees via tree-sitter, extract node-type sequences (67 types), and encode them with a bidirectional LSTM: $h_{\text{ast}} \in \mathbb{R}^{128}$. A parallel MLP encodes 22 handcrafted structural features (indentation patterns, naming conventions, complexity proxies): $h_{\text{struct}} \in \mathbb{R}^{128}$.

Spectral stream. Treating token ID sequences as 1D signals, we apply FFT at four window sizes $\{32, 64, 128, 256\}$ and extract 64 spectral features (band energies, spectral centroid, rolloff, flatness). An MLP produces $h_{\text{spec}} \in \mathbb{R}^{128}$. This stream captures periodic artifacts introduced by LLM decoding strategies.

Fusion. The token, AST, and structural representations are fused via 4-head cross-attention followed by a gated projection into $h_{\text{neural}} \in \mathbb{R}^{512}$. A learned gating network then combines the neural and spectral predictions:

$$\hat{y} = \sigma(g([h_{\text{neural}}; h_{\text{spec}}]))_0 \cdot f_{\text{n}}(h_{\text{neural}}) + \sigma(g(\cdot))_1 \cdot f_{\text{s}}(h_{\text{spec}}) \tag{1}$$

3.2 Hierarchical Affinity Loss

The key observation is that LLMs form a family tree: Nxcode descends from Qwen1.5, CodeLlama from Llama, Gemini and Gemma share a Google lineage. We define a mapping $\phi : \mathcal{C} \to \mathcal{F}$ from class labels to family indices:

Family	Members	Relationship
Human	Human	—
Meta	Llama3.1, CodeLlama	CodeLlama fine-tuned from Llama
GPT	GPT-4o	—
Qwen	Qwen1.5, Nxcode	Nxcode fine-tuned from Qwen

Given a batch of ℓ_2-normalized embeddings $\{\bar{h}_i\}$ with cosine distances $d_{ij} = 1 - \bar{h}_i^\top \bar{h}_j$, we perform *batch-hard triplet mining*:

$$\mathcal{L}_{\text{hier}} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \big[\, d_{i,p^*} - d_{i,n^*} + \alpha \,\big]_+ \tag{2}$$

where $p^* = \arg\max_{j:\phi(y_j)=\phi(y_i)} d_{ij}$ is the hardest same-family sample, $n^* = \arg\min_{j:\phi(y_j)\neq\phi(y_i)} d_{ij}$ is the closest different-family sample, and $\alpha = 0.3$ is the margin. This encourages the model to learn an embedding space where family structure is respected but intra-family members remain distinguishable.

Table 2: Author attribution on CoDET-M4 (6-class: Human, CodeLlama, GPT-4o, Llama3.1, Nxcode, Qwen1.5). Macro-averaged P/R/F1/Acc (%). ΔF1 is improvement over UniXcoder.

Method	Prec.	Recall	F1	Acc.	ΔF1
SVM	29.10	28.51	27.63	49.70	−38.70
CatBoost	50.46	44.41	45.42	66.19	−20.91
CodeBERT	63.14	68.10	64.80	77.65	−1.53
CodeT5	62.67	69.40	62.45	78.25	−3.88
UniXcoder	64.80	69.54	66.33	79.35	—
SpectralCode (ours, no $\mathcal{L}_{\text{hier}}$)	—	—	69.82	—	+3.49
HierTreeCode (ours)	—	—	**70.55**	—	**+4.22**

The family tree adapts to each benchmark: for AICDBench T2 (12 classes), we group Google models (Gemma + Gemini) and Chinese AI labs (DeepSeek + Qwen); for DroidCollection, we define a generation-type hierarchy (human → refined → machine → adversarial).

Total loss:

$$\mathcal{L} = \mathcal{L}_{\text{focal}} + 0.3\,\mathcal{L}_{\text{neural}} + 0.3\,\mathcal{L}_{\text{spec}} + 0.4\,\mathcal{L}_{\text{hier}} \tag{3}$$

where auxiliary losses on individual heads ensure both paths learn useful representations independently.

4 Experiments

4.1 Setup

Benchmarks. We evaluate on CoDET-M4 (500K, 3 languages, 5 generators), AICDBench (2.05M, 9 languages, 77 models), and DroidCollection (1.06M, refined + adversarial code). See Section 2 and Appendix A for details.

Implementation. ModernBERT-base backbone, AdamW (2×10^{-5} encoder, 10^{-4} heads), cosine schedule with 10% warmup, batch size 64, BF16 on H100, 3 epochs. Focal loss with $\gamma = 2.0$ and inverse-frequency class weights.

Baselines. We compare against all baselines from Orel et al. [2025]: Fast-DetectGPT (zero-shot), SVM, CatBoost, CodeBERT [Feng et al., 2020], CodeT5 [Wang et al., 2021], and UniXcoder [Guo et al., 2022]. We also ablate against SpectralCode (our architecture without $\mathcal{L}_{\text{hier}}$).

4.2 Author Attribution: The Main Event

Table 2 presents the core result. HierTreeCode achieves 70.55% macro-F1 on 6-class attribution, surpassing UniXcoder by **+4.22 points** and CodeBERT by **+5.75 points**. The gain over SpectralCode (+0.73) isolates the contribution of the Hierarchical Affinity Loss.

4.3 Where the Gains Come From: Per-Generator Analysis

The aggregate numbers mask a striking pattern: the gains are concentrated on the *genealogically confused* classes. Table 3 shows per-generator F1. GPT-4o (a singleton family) and Human are easily distinguished at >87% F1. The bottleneck is the Qwen family: Qwen1.5 achieves only 0.44 F1 even with HierTreeCode, and Nxcode reaches 0.59. The Hierarchical Affinity Loss provides the largest improvement on Qwen1.5 (+3.02 points over baseline), confirming that family-aware constraints target the right failure mode.

4.4 Robustness Across Languages and Sources

Table 4 breaks down author attribution by language and code source. Two findings stand out: (1) performance is relatively stable across languages (69–72% F1), but (2) *code source* creates dramatic variation. CodeForces (competitive programming) achieves 77.17%, while GitHub (diverse real-world code) drops to 56.18%—a 21-point gap. This suggests that code domain diversity, not programming language, is the primary confound for attribution.

Table 3: Per-generator F1 on CoDET-M4 author attribution. The Qwen family (Qwen1.5 + Nxcode) is the dominant bottleneck. HierTreeCode provides the largest improvement on Qwen1.5.

	Human	**GPT-4o**	**Llama3.1**	**CodeLlama**	**Qwen1.5**	**Nxcode**
SpectralCode	0.86	0.91	0.66	0.73	0.41	0.58
HierTreeCode	**0.87**	**0.92**	**0.67**	**0.74**	**0.44**	**0.59**
Δ	+.01	+.01	+.01	+.01	**+.03**	+.01

Table 4: HierTreeCode author attribution (6-class F1, %) broken down by language and source.

By Language			**By Source**		
C++	**Java**	**Python**	**CodeForces**	**GitHub**	**LeetCode**
72.31	70.12	69.22	77.17	56.18	60.35

4.5 Out-of-Distribution: Unseen Languages

Table 5 evaluates binary detection on languages never seen during training, following the protocol of Orel et al. [2025]. UniXcoder generalizes remarkably well to C#, Go, and PHP (88–96% F1), but struggles with JavaScript (81.48%). HierTreeCode maintains competitive OOD performance while additionally solving the attribution task that OOD binary baselines cannot address.

4.6 DroidCollection: Refined and Adversarial Detection

Table 6 places our method in context with baselines from the DroidCollection benchmark [DROID, 2025]. On the 3-class task (human/machine/refined), SpectralCode achieves 84.73% macro-F1—**surpassing all zero-shot detectors** (Fast-DetectGPT 64.54%, GPTZero 49.10%), **all fine-tuned baselines** (CatBoost 72.31%), and competitive with full-training methods (GPT-SnifferFT 82.75%, CoDet-M4FT 83.25%). On the harder 4-class task that separates adversarial (DPO-tuned) code, SpectralCode maintains 84.67% macro-F1.

This is notable because SpectralCode uses a *general-purpose* architecture not specifically designed for the refined/adversarial distinction, yet matches dedicated detectors trained with domain-specific losses. We expect HierTreeCode's family-aware constraints to further improve on multi-class tasks where generation-type hierarchy is defined.

4.7 AICDBench: The OOD Frontier

Table 7 shows our results on AICDBench alongside SpectralCode as ablation. On T2 (12-class family attribution) and T3 (4-class fine-grained), the Hierarchical Affinity Loss is active with benchmark-specific family trees.

The AICD T1 collapse. All methods—including ours—exhibit severe OOD degradation on AICD T1 (val 99.5% $\rightarrow$ test 29.8%). This reflects a fundamental distribution shift in AICD's progressive test splits (unseen languages + unseen domains simultaneously), and suggests that language-level domain adaptation—orthogonal to family-aware losses—is needed for this benchmark.

4.8 Ablation: What Matters?

We ablate HierTreeCode on CoDET-M4 author attribution (Table 8). The Hierarchical Affinity Loss contributes +0.73 F1, making it the single most impactful addition. The spectral stream contributes +1.05 and the AST encoder +1.35, confirming that multi-stream fusion captures complementary signals. Notably, using simple concatenation instead of cross-attention degrades F1 by 1.15 points.

Failed approach: domain adversarial training. We also experimented with gradient reversal to learn generator-invariant features (DANN-style). This *catastrophically* degrades attribution F1 from 69.82 to 62.89 (-6.93), because the adversarial objective directly contradicts the attribution goal: making generators indistinguishable is the opposite of what we want. This negative result confirms that naive domain adaptation is harmful for attribution tasks.

Table 5: Binary detection on unseen languages (OOD, CoDET-M4 Table 10 protocol). Models trained on {C++, Java, Python}, tested on held-out languages. F1 (%).

Method	C#	Go	JavaScript	PHP	Avg.
CatBoost	51.01	64.72	26.03	45.08	46.71
CodeBERT	45.31	52.46	26.84	56.68	45.32
CodeT5	78.55	88.78	34.81	93.66	73.95
UniXcoder	**91.31**	**90.01**	**81.48**	**96.17**	**89.74**

Table 6: Comparison with DroidCollection baselines. Paper baselines report weighted-F1 (3-class domain avg.); our methods report macro-F1. †: $\mathcal{L}_{\text{hier}}$ active. Despite the metric difference (macro-F1 $\leq$ weighted-F1 for imbalanced data), our methods surpass most baselines.

Category	Method	3-class	4-class
Zero-shot	Fast-DetectGPT	64.54^{w}	—
	GPTZero	49.10^{w}	—
	GPTSniffer	38.95^{w}	—
Fine-tuned	CatBoost	72.31^{w}	—
	GCN	51.63^{w}	—
Full training	GPT-SnifferFT	82.75^{w}	—
	CoDet-M4FT	83.25^{w}	—
	DroidDetect-Base	$\mathbf{86.76}^{w}$	$\mathbf{92.95}^{w}$
	DroidDetect-Large	$\mathbf{88.78}^{w}$	$\mathbf{94.30}^{w}$
Ours	SpectralCode	84.73^{m}	84.67^{m}
	HierTreeCode†	TBD	TBD

^{w} weighted-F1 (from paper), ^{m} macro-F1 (our evaluation). Macro-F1 is stricter for imbalanced classes.

5 Discussion

The story beyond numbers. Our results tell a consistent story across three benchmarks: binary detection is solved, but the harder problems of attribution, OOD generalization, and adversarial robustness remain open. HierTreeCode makes targeted progress on attribution by exploiting the one piece of structure that classifiers currently ignore—LLM genealogy. The gains are modest in aggregate (+0.73 F1 from the hierarchical loss alone) but concentrated on the hardest failure modes (Qwen1.5 +3.02 F1), which is precisely where improvement matters most.

Limitations. (1) The family tree requires knowledge of model relationships, which may be unavailable for closed-source models. We discuss automatic tree discovery from embedding structure in Appendix B. (2) The spectral path uses CPU-bound FFT, limiting throughput. (3) AICD T1 OOD collapse remains unsolved—family-aware losses do not address language-level distribution shift. (4) While we improve Qwen1.5 F1 from 0.41 to 0.44, this remains far from acceptable for deployment; the genealogical confusion problem is mitigated, not solved.

Broader impact. Code attribution supports software provenance, academic integrity, and AI safety auditing. We note the dual-use risk: attribution tools could surveil developers or unfairly penalize legitimate AI-assisted coding. We advocate for deployment with transparency and consent.

6 Conclusion

We have argued that AI code detection must move beyond binary classification. The harder problems—authorship attribution, OOD generalization, and adversarial robustness—demand methods that exploit structural knowledge about the code generation landscape. HierTreeCode takes a first step by encoding LLM genealogy as hierarchical distance constraints, achieving state-of-the-art attribution on CoDET-M4 while identifying key challenges (source-domain sensitivity, OOD collapse) that the community must address next. The family tree of LLMs will only grow; detection methods must grow with it.

Table 7: AICDBench results (macro-F1). †: $\mathcal{L}_{\text{hier}}$ active.

	T1 (2-cls OOD)	**T2**† (12-cls family)	**T3**† (4-cls fine-gr.)
SpectralCode	29.83	18.93	56.31
HierTreeCode	29.83	TBD	TBD

Table 8: Ablation on CoDET-M4 author attribution (6-class macro-F1, %).

Configuration	**F1**	**Δ**
Full HierTreeCode	**70.55**	—
w/o $\mathcal{L}_{\text{hier}}$ (= SpectralCode)	69.82	−0.73
w/o spectral path	69.50	−1.05
w/o AST encoder	69.20	−1.35
w/o structural features	69.65	−0.90
concat instead of cross-attn	69.40	−1.15

References

Orel, D., Azizov, E., and Nakov, P. CoDET-M4: A multi-generator benchmark for AI-generated code detection. In *Findings of ACL*, 2025.

AICD-Bench Team. AICDBench: A challenging benchmark for AI-generated code detection. *arXiv preprint*, 2025.

Project DROID. Droid: A resource suite for AI-generated code detection. *arXiv preprint*, 2025.

Feng, Z., Guo, D., Tang, D., et al. CodeBERT: A pre-trained model for programming and natural languages. In *EMNLP Findings*, 2020.

Wang, Y., Wang, W., Joty, S., and Hoi, S. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *EMNLP*, 2021.

Guo, D., Lu, S., Duan, N., et al. UniXcoder: Unified cross-modal pre-training for code representation. In *ACL*, 2022.

Caliskan-Islam, A., et al. De-anonymizing programmers via code stylometry. In *USENIX Security*, 2015.

Schroff, F., Kalenichenko, D., and Philbin, J. FaceNet: A unified embedding for face recognition and clustering. In *CVPR*, 2015.

Wang, H., et al. CosFace: Large margin cosine loss for deep face recognition. In *CVPR*, 2018.

Deng, J., Guo, J., Xue, N., and Zafeiriou, S. ArcFace: Additive angular margin loss for deep face recognition. In *CVPR*, 2019.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. Focal loss for dense object detection. In *ICCV*, 2017.

Frank, J., et al. Leveraging frequency analysis for deep fake image recognition. In *ICML*, 2020.

Answer.AI. ModernBERT: A modern BERT for the modern era. *arXiv preprint*, 2024.

DETree Authors. DETree: Hierarchical decision tree for AI text detection. *arXiv:2510.17489*, 2025.

Snell, J., Swersky, K., and Zemel, R. Prototypical networks for few-shot learning. In *NeurIPS*, 2017.

Sagawa, S., Koh, P. W., Hashimoto, T. B., and Liang, P. Distributionally robust neural networks for group shifts. In *ICLR*, 2020.

A Benchmark Details

CoDET-M4. 500K code samples in Python, Java, C++ from CodeForces, GitHub, and LeetCode. Generators: CodeLlama, GPT-4o, Llama3.1, Nxcode, Qwen1.5. Split: 100K train, 20K val, 47.7K test.

AICDBench. 2.05M samples across 9 languages (Python, Java, C++, C, Go, PHP, C#, JS, Rust) from 77 models in 11 families. Three tasks: T1 (binary, 4-split progressive OOD), T2 (12-class family attribution), T3 (4-class: human/machine/hybrid/adversarial).

DroidCollection. 1.06M samples across 7 languages from 43 models. Includes machine-refined code (3 scenarios: continuation, gap-fill, rewrite) and 157K DPO-tuned adversarial examples. Tasks: T3 (3-class), T4 (4-class with adversarial).

B Automatic Family Tree Discovery

When model genealogy is unknown, we propose learning the family tree from embedding structure. After training without $\mathcal{L}_{\text{hier}}$, we compute class centroids in embedding space and apply hierarchical agglomerative clustering. The resulting dendrogram can serve as the family tree for a second training round. We leave full evaluation of this approach to future work.

C Hyperparameter Sensitivity

Table 9: Sensitivity of λ_{hier} and margin α on CoDET-M4 author F1 (%).

λ_{hier}	α	**Author F1**
0.0	—	69.82
0.1	0.3	70.10
0.2	0.3	70.30
0.4	**0.3**	**70.55**
0.6	0.3	70.40
0.4	0.1	70.25
0.4	0.5	70.45

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a

proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.